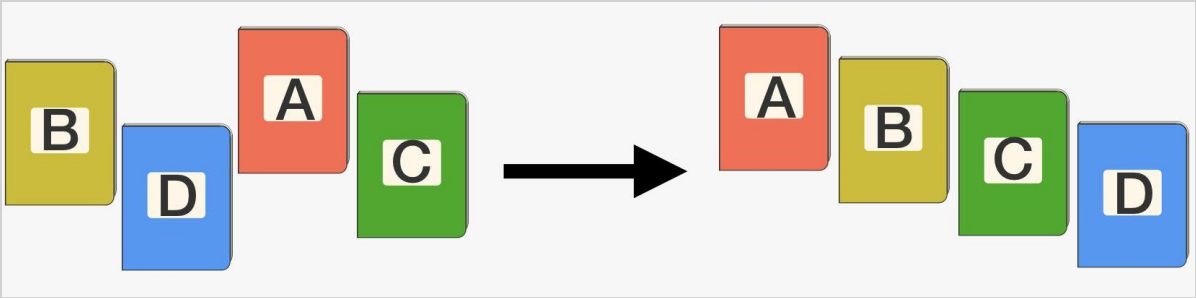
Bubble Sort

MODULE 3

Sorting

- Sorting is arranging the elements in ascending and descending order

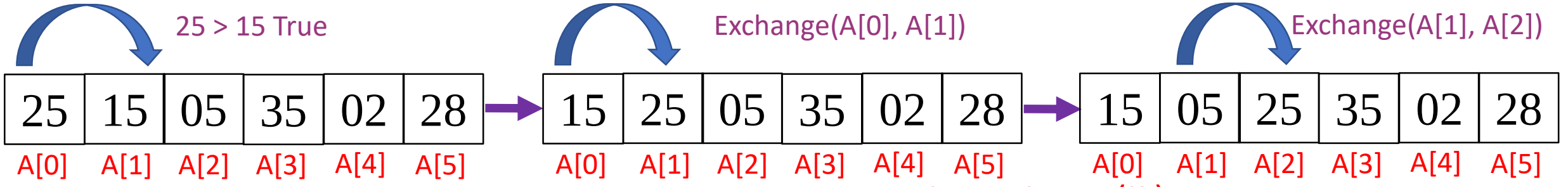


Procedure to perform Bubble sort

- Let us an Array A of N elements is in memory.
- In the bubble sort, consecutive adjacent pairs of elements in the array are compared with each other. If the element at the lower Index is greater than the element at the higher Index, the two elements are interchanged so that the element is placed before the bigger one. (for sorting into ascending order).
- The process will continue till the list of unsorted elements exhausts.

This is the algorithm, now you carry on

```
Bubble_Sort(A,N)
step 1: Repeat step 2 for i: = 0 to N-1
step 2:       Repeat For j:=0 to N-i-1
step 3:             IF  A[J] > A[J+1] THEN
                                    Exchange(A[J], A[J+1])
                End of Inner For loop
           End outer for loop
step 4: Exit.
```



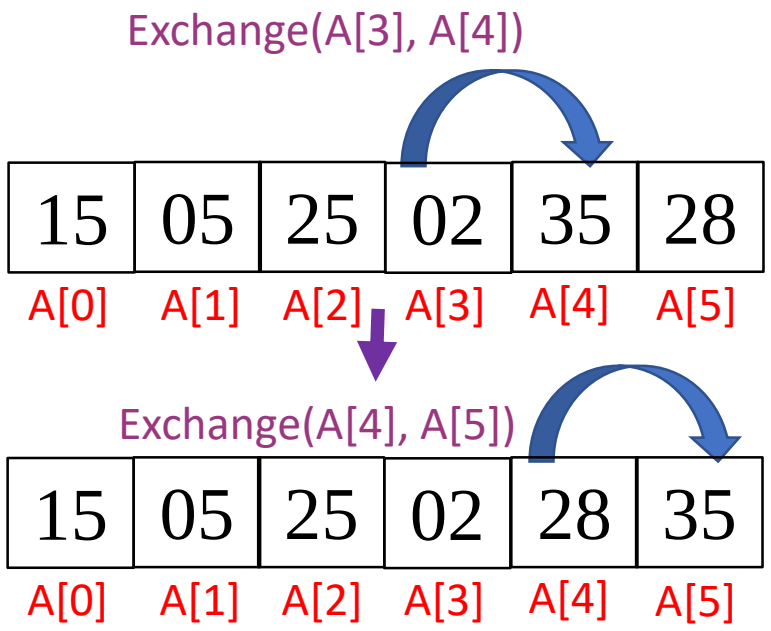
Outer loop (OL)

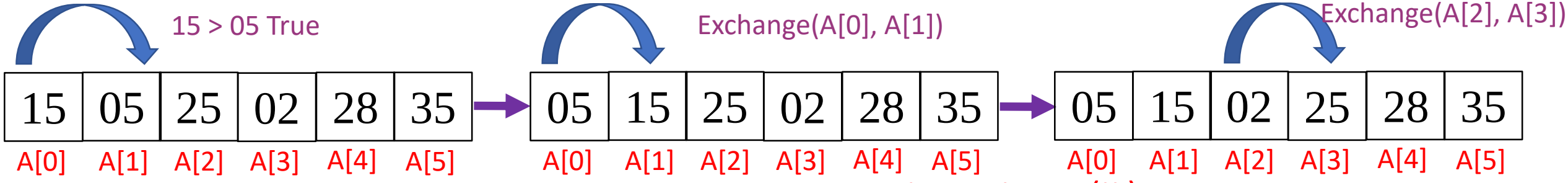
Inner Loop (IL)

OL

Pass 0

i=0	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
0	0 <=5 T	0	0 < 6-0-1 0 < 5 T	A[0] > A[1] 25 > 15 T	Exchange(25,15)	J=0+1=1	-
		-	1 < 5 T	A[1] > A[2] 25 > 05 T	Exchange(25,05)	J=1+1=2	-
		-	2 < 5 T	A[2] > A[3] 25 > 35 F	--	J=2+1=3	-
		-	3 < 5 T	A[3] > A[4] 35 > 02 T	Exchange(35,02)	J=3+1=4	-
		-	4 < 5 T	A[4] > A[5] 35 > 28 T	Exchange(35,28)	J=4+1=5	-
		-	5 < 5 F	-	-	-	1





Outer loop (OL)

Inner Loop (IL)

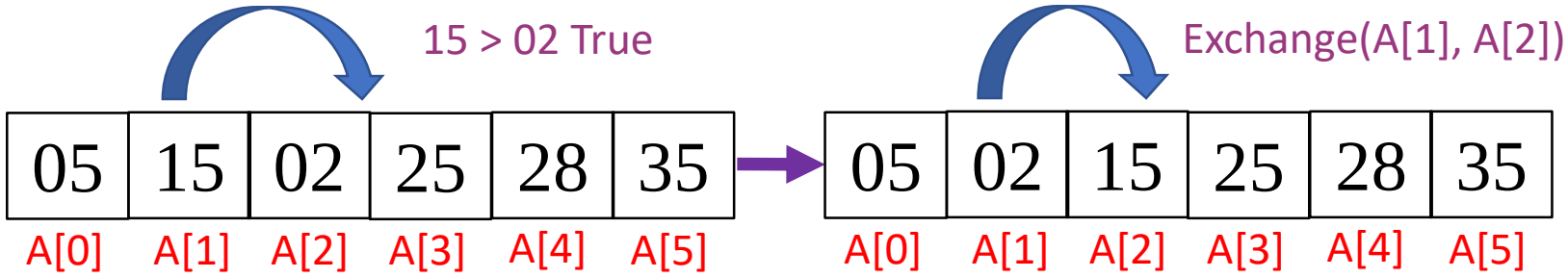
OL

Pass 1

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
1	1 <=5 T	0	0 < 6-1-1 0 < 4 T	A[0] > A[1] 15 > 05 T	Exchange(A[0],A[1]) Exchange(15,05)	J=0+1=1	-
-		-	1 < 4 T	A[1] > A[2] 15 > 25 F	-	J=1+1=2	-
-		-	2 < 4 T	A[2] > A[3] 25 > 02 T	Exchange(A[2],A[3]) Exchange(25,02)	J=2+1=3	-
-		-	3 < 4 T	A[3] > A[4] 25 > 28 F	--	J=3+1=4	-
-		-	4 < 4 F	-	-	-	2

```
Bubble_Sort(A,N)
step 1: Repeat step 2 for i: = 0 to N-1
step 2:   Repeat For j:=0 to N-i-1
step 3:     IF  A[J] > A[J+1] THEN
                Exchange(A[J], A[J+1])
            End of Inner For loop
        End outer for loop
step 4: Exit.
```

Figures shows only when exchange happens in the Inner loop whereas Table shows complete tracing of the algorithm



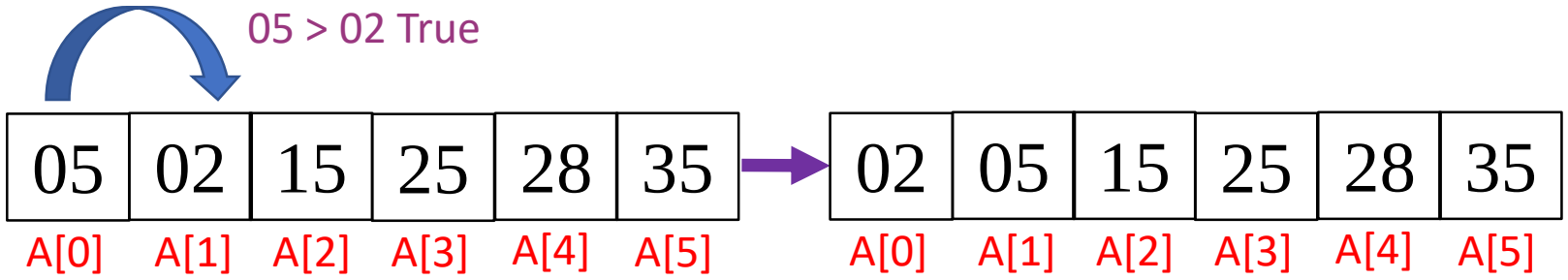
Outer loop (OL)

Inner Loop (IL)

OL

Pass 2

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
2	2 <=5 T	0	0 < 6-2-1 0 < 3T	A[0] > A[1] 05 > 15 F	-	J=0+1=1	-
		-	1 < 3 T	A[1] > A[2] 15 > 02 T	Exchange(A[1], A[2]) Exchange(15, 02)	J=1+1=2	-
		-	2 < 3 T	A[2] > A[3] 15 > 25 F	--	J=2+1=3	-
		-	3 < 3 F	--	--	--	3



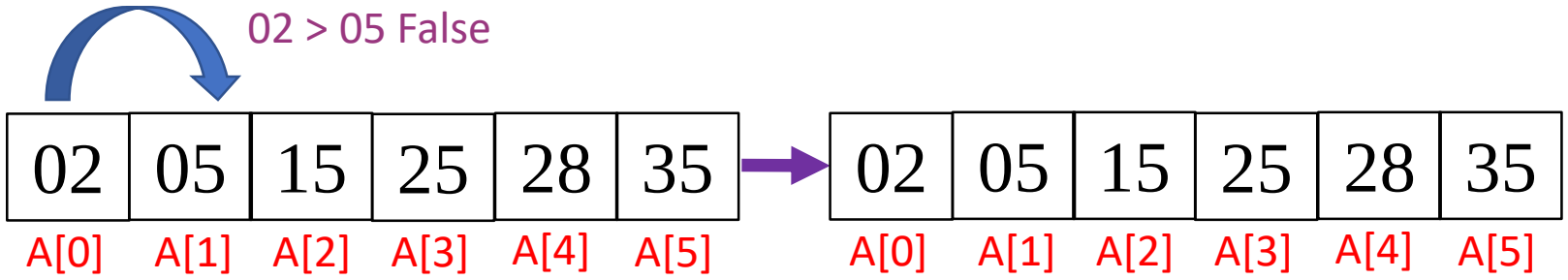
Outer loop (OL)

Inner Loop (IL)

OL

Pass 3

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
3	3 <=5 T	0	0 < 6-3-1 0 < 2T	A[0] > A[1] 05 > 02 T	Exchange(A[0], A[1]) Exchange(05, 02)	J=0+1=1	-
		-	1 < 2 T	A[1] > A[2] 05 > 15 F	--	J=1+1=2	-
		-	2 < 2 F	--	--	--	4



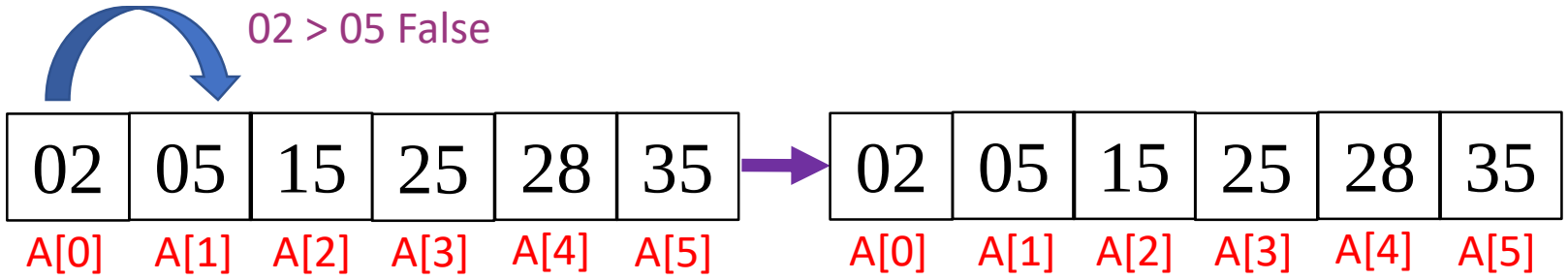
Outer loop (OL)

Inner Loop (IL)

OL

Pass 4

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
4	4 <=5 T	0	0 < 6-4-1 0 < 1 T	A[0] > A[1] 02 > 05 F	--	J=0+1=1	-
		-	1 < 1 F	--	--		5



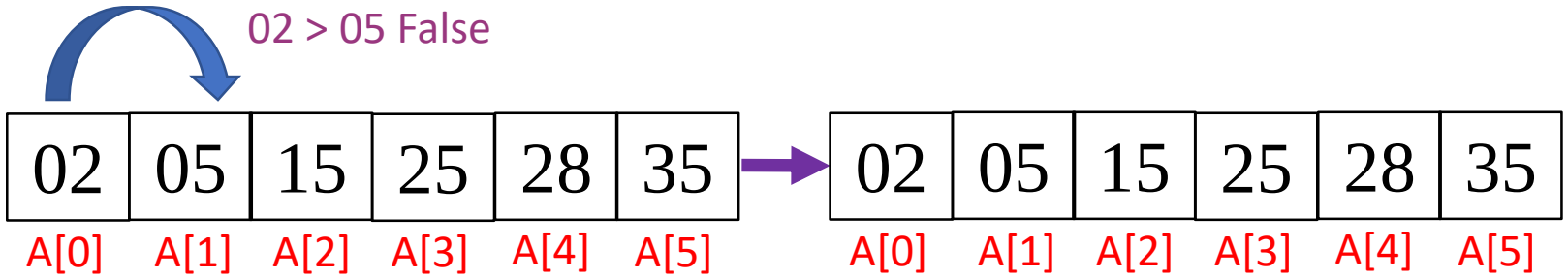
Outer loop (OL)

Inner Loop (IL)

OL

Pass 5

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
5	5 <=5 T	0	0 < 6-5-1 0 < 0 F	--	--	--	6



Outer loop (OL)

Inner Loop (IL)

OL

Pass 6

i	i<=N-1	j=0	j < N-i-1	If (A[j] > A[j+1])	Exchange (A[j], A[j+1])	J=J+1	i++
6	6 <=5 F	-	--	--	--	--	--

Time Complexity

- The complexity of any sorting algorithm depends upon the number of comparisons.
- In bubble sort, we have total n-1 number of passes. In the first pass n-1 comparisons made to place highest element in its correct position. Then in pass 2, there are n-2 comparisons and second highest element is placed in its position. Therefore, to compute the time complexity of bubble sort, we have to calculate total number of comparisons.

They are, $f(n) = (n-1) + (n-2) + \cdots + 3 + 2 + 1$

$= \frac{n(n-1)}{2}$

$= O(n^2).$

End